

Claim Denial Prediction – Write-Up

EHP AI/ML Take-Home – Predicting claim denials **before** submission so a front-end review team can act within its limited review capacity.

1. How I framed the problem – which errors matter more, and why

This is not "classify every claim correctly." It is a **ranking-and-triage** problem shaped by one hard operational constraint: **the review team can only inspect the top 25% of claims by risk score before submission**. The model's job is to make sure that as many true denials as possible land inside that top 25%.

That constraint decides which errors matter:

- **False negatives** (a claim that **will** be denied but the model ranks low, so it never reaches a reviewer) are the expensive error. It sails through, gets denied by the payer, and the hospital eats the rework — investigate, correct, resubmit. **Recall inside the review budget is what we protect.**
- **False positives** (a clean claim that gets flagged) cost only a few minutes of reviewer time. Tolerable, but not free — if precision is too low, reviewers waste their limited capacity chasing non-issues.

Because denials are only ~22% of claims, **accuracy is the wrong metric** — a model that predicts "will be paid" for everything scores ~78% accuracy while catching zero denials. I therefore optimized and reported:

- **PR-AUC (average precision)** — overall ranking quality on the rare class.
 - **Precision@top-25%** and **Recall@top-25%** — performance at the exact operating point the review team will actually use.
-

2. Data findings worth sharing with a non-technical manager

1. **Most denials are avoidable paperwork gaps, not random.** When a claim needed prior authorization but none was on file, its denial rate **jumped from 19% to 46%** — more than double. Missing documentation (18% → 39%) and unverified patient eligibility (19% → 36%) showed the same pattern. **The biggest denial risks are things a reviewer can fix before submission.**

2. **A handful of simple checks explain most of the risk.** The strongest signals are all "did we complete a required step?" flags — authorization, documentation, referral, eligibility. The dollar amount of the claim and the calendar month barely mattered. This means the review team can be handed a **short, concrete checklist**, not a black box.

3. **The denial rate is stable over time but the data periods don't overlap.** The historical claims are all from 2024; the current claims to score are from early 2025. There's no meaningful month-to-month trend (calendar month correlates with denial at ~0.02), so seasonality isn't a driver — but it does mean we should avoid features that memorize specific months.

3. What I built, baselines compared, and threshold choice

Pipeline (modular, reproducible): feature engineering → preprocessing (ColumnTransformer: scale numerics, one-hot categoricals, passthrough flags) → classifier, all inside a single scikit-learn Pipeline. Engineered features encode the **gaps** that drive denials (auth_gap, referral_gap, out_of_network, eligibility_unverified, billed_to_expected_ratio). Leakage/identifier columns (claim_id, denial_reason, split, service_month) are excluded.

Models compared (all with class-imbalance handling, all tuned with 5-fold GridSearchCV on PR-AUC):

Model	Validation PR-AUC	Notes
Logistic Regression	0.454	Winner – stable, interpretable
XGBoost (tuned)	0.444	Overfits: train 0.56 → val 0.44
Random Forest (tuned)	0.432	Overfits: train 0.60 → val 0.43

Why LogReg won: the denial signal is essentially **linear and additive** (a few binary gap flags add up to risk), and the training set is small (~2.1k rows). Tree ensembles memorized the training data (train PR-AUC ~0.56–0.60) but did not generalize better than LogReg on validation/test. The simplest model with the smallest train-validation gap was the right choice.

Threshold choice: I set the operating threshold to the probability cutoff that selects the **top 25% of validation claims by risk** (~0.59), frozen on validation to avoid leakage. This directly encodes the review team's capacity: everything at or above that cutoff is "High" risk (flagged for review).

Risk tiers (documented in the README):

- High = $p \geq \text{threshold}$
- Medium = $0.5 \times \text{threshold} \leq p < \text{threshold}$
- Low = $p < 0.5 \times \text{threshold}$

4. Test-set metrics and denial capture at top 25%

Evaluated on the held-out **test split (539 claims, 26% denials)** using the model and threshold frozen during training.

Metric	Test value
ROC-AUC	0.694
PR-AUC	0.505
Precision @ top-25%	47.4%
Recall @ top-25% (denial capture)	~46%

In plain terms: if the review team inspects only the **top 25% of claims by risk (135 of 539)**, they would **catch ~46% of all denials** — roughly half of every preventable denial surfaced within a quarter of the workload. Of the claims they review, **~47% are genuine denial risks** (versus a 26% base rate), so the model nearly **doubles the hit**

rate of random review.

5. LLM prompt, one example explanation, and one honest limitation

System prompt (grounded, plain-language, action-oriented)

You are a claims-denial prevention assistant for a hospital revenue-cycle team.

Given a claim's risk factors, write a 2–3 sentence explanation an analyst can act on in seconds.

Rules:

1. Ground every statement **only** in the provided field values and risk factors — do not invent facts.
2. Use plain language, no jargon.
3. Include exactly one specific recommended action.
4. Make clear this is a **risk estimate**, not a guarantee of denial.
5. If the risk is low, say so plainly and note no urgent action is needed.

Each claim's user prompt injects its actual field values (payer type, visit type, billed amount, days to submit) and the model-derived top risk factors.

Example explanation (highest-risk current claim, CCLM-00082, 95% risk)

This claim has a high estimated denial risk (95%), driven mainly by: referral required but not present. Recommended action: secure the referral document before submitting. This is a risk estimate, not a guarantee the claim will be denied.

Sanity check on a low-risk claim

This claim shows a low estimated denial risk (4%) with no dominant red flags. No urgent action is needed before submission. This is a risk estimate, not a guarantee of the final outcome.

Honest limitation

The explanations are built from **rule-based risk factors** that mirror the model's known drivers, not from a per-claim attribution of the model itself (e.g., SHAP). When a claim scores high due to a **combination** of weaker signals rather than one dominant flag, the "top risk factor" shown may be a reasonable proxy rather than the mathematically exact top contributor. The text is faithful to the claim's data, but it explains **the domain rules**, not the model's internal weightings claim-by-claim.

6. What I would improve with more time

- **Per-claim model attribution (SHAP):** replace rule-based `top_risk_factors` with true per-prediction contributions, so explanations reflect exactly why **this** claim scored as it did, including combination effects.
- **Calibration:** apply probability calibration (Platt/isotonic) so `denial_probability` reads as a trustworthy probability, and add a cost-sensitive threshold that weighs rework cost vs. reviewer time explicitly.
- **More data / richer features:** ~2.1k training rows caps model complexity. With more history, I'd revisit gradient boosting, add payer-specific denial-rate features (target-encoded with proper CV), and test interaction terms.
- **Temporal validation:** since scoring data is from a later period than training, I'd add a time-based backtest and monitor for drift (payer policy changes shift denial patterns over time).
- **Feedback loop:** capture reviewer outcomes (was the flagged claim actually fixed and paid?) to retrain and measure real prevented-denial value, not just offline metrics.
- **Human-in-the-loop guardrails on the LLM:** add automatic checks that every generated explanation cites only fields present in the claim, flagging any hallucinated content before it reaches an analyst.